

Software Architecture and Design Thinking

116U01C701

Module 1

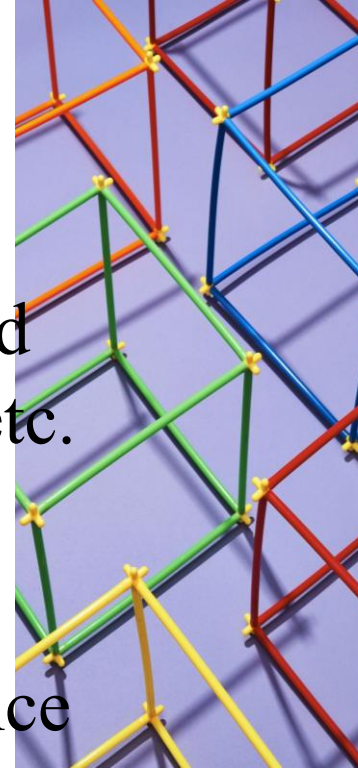
Module 1 (12)

1. Introduction to Software Architecture

- 1.1 Basic Concepts: Elements of an Architecture Design, Designing Architectures, Conventional Architectural styles : Concepts of Software Architecture, Models, Processes, Stakeholders**
- 1.2 Creating An Architecture Design Styles and Architectural Patterns, Pipes and Filters, Event-based, Implicit Invocation**
- 1.3 Layered Systems, Repositories, Interpreters**
- 1.4 Architectural Design process**

Architecture

- The science, art, or profession of designing and constructing buildings, or buildings, bridges, etc.
- Software Architecture:
the internal organization of a computer's components with particular reference to the way in which data is transmitted
- The arrangement of the various devices in a complete computer system or network



General Perspective

- **Civil Engineer:** Basic principles related to construction engineering : Focuses on Functional Requirements
- **Architect:** Appearance, usability, aesthetically pleasing,
- usability : Focuses on Non-Functional Requirements
- Architect will choose material to enhance the functionality/ usability of finished product.
- **Ever Enhancing, changes with time**

Architecture focuses

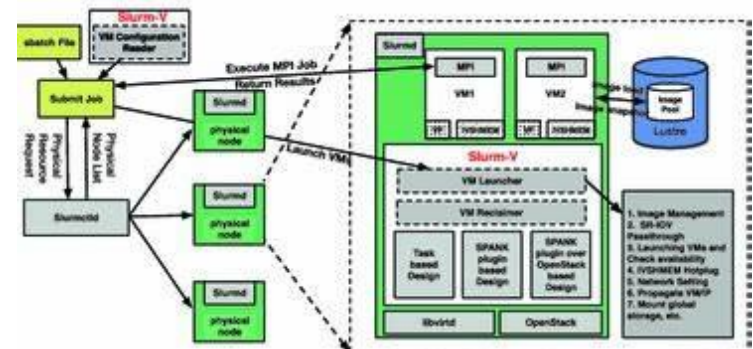


- **More on Non Functional Attributes**
 - Functional Requirements are THE MUST to be satisfied
-
- Non Functional are add-ons: when provided will
 - **For developers**
 - Ease to develop
 - Modify
 - Maintain
 - Adapt to accommodate changing requirements
 - Scalable
 - **For end users**
 - Secure
 - User friendly

Software Architecture

Definition:

“the set of principal design decisions made about the system.”

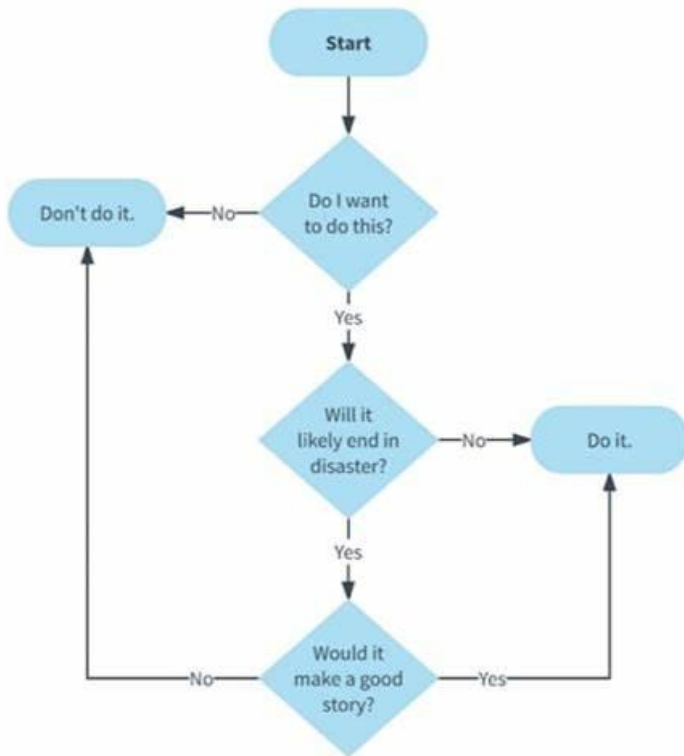


Kind of decisions to be made

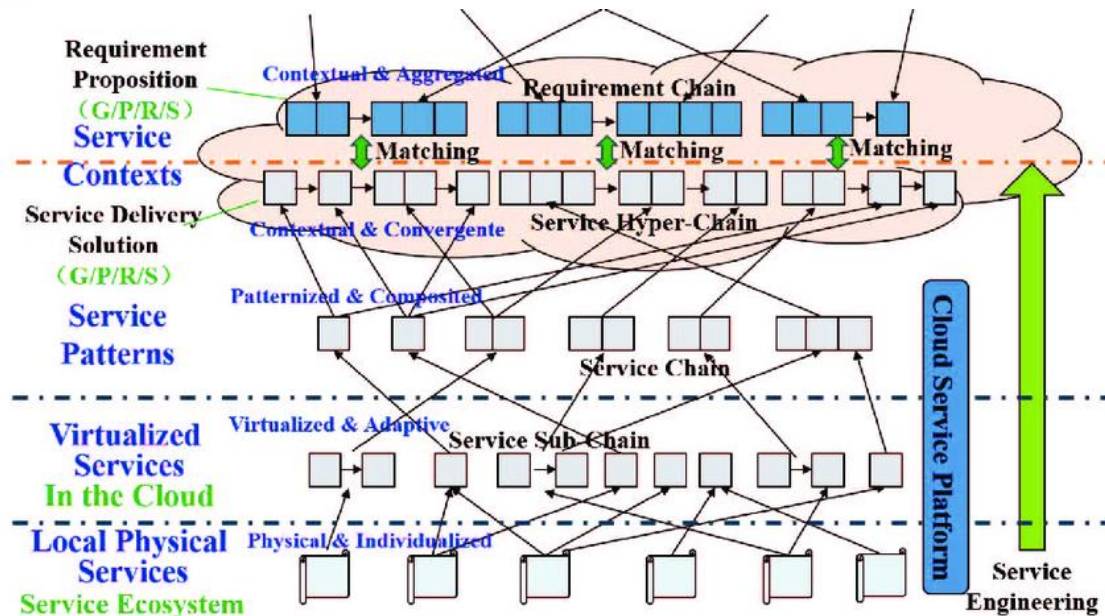
- Include from high level architectural choices to low level implementation details
-
- Will have impact on **system's success, performance, maintainability, scalability, and security**
 - Require understanding **trade-offs** and their long-term implications.

Introduction

Simple System



Complex System



Introduction

As the size and complexity of software systems increase, the design and specifications of **overall system structure** become **more significant issues than the choice of algorithms** and data structures of computation.

Basic Concepts

Architecture:

- A software system's architecture is the set of important design designs made about the system
- A reference architecture is the set of important decisions that are simultaneously applicable to multiple related systems, typically within an application domain, with explicitly defined point of variation

Basic Concepts

Architecture decisions:

- Will depend on system's goal
- System's stakeholders
- Some non-technical considerations

Selection of algorithm *is NOT principal decision*

Basic Concepts

Design decisions are about:

- **System structure:** organization and composition of various building blocks
- **Functional behavior:** performance of data processing, storage and visualization; responsibility assignment to each building block
- **Interaction:** style/ protocol of communication between various building blocks
- **Non-functional properties:** Quality related such as reusability, correctness, efficiency etc.
- **Implementation:** Language, framework to be used for implementation / coding

Need of Architectural Description Language (ADL)

- Building Complex Systems with Clarity and Control
- Multiple stakeholders: end users, developers, tester etc. involved in software system development
- ALL of them should be on the same page
- Very complex system to be developed

Challenges in development of a system

- Modern systems are incredibly complex.
- Thousands of interacting components
- Diverse technologies and platforms
- Evolving requirements and environments
- **Without clear definitions, managing this complexity becomes a monumental task, leading to errors and delays.**

ADL

- A **formal language** used to describe the **architecture of a system**.
- It focuses on:
 - **Components:** Independent units of computation or data.
 - **Connectors:** Mechanisms for interaction between components.
 - **Configurations:** How components and connectors are assembled.
- Provides a **higher level of abstraction** than programming languages.

ADL

- **Task 1: Manages and Handling Complexity**
 - Provides **structured abstraction**, allowing architects to focus on essential elements and their relationships.
 - Helps **decompose** complex systems into manageable parts.
 - Makes the system's overall structure comprehensible

ADL

- **Task 2: Enhance Communication and Collaboration**
- Ambiguity and misinterpretations arise from informal descriptions (e.g., whiteboard diagrams, natural language).
 - Establishes a **common, unambiguous language and notation** for all stakeholders.
 - Facilitates clear and precise communication between business, development, and operations teams.
 - Reduces misunderstandings and costly rework.

ADL

- **Task 3: Enable Analysis and Verification**
- Design flaws, performance bottlenecks, or security vulnerabilities are often discovered late, making them expensive to fix.
 - Many ADLs support **formal or semi-formal analysis**.
 - Allows for **early detection of design flaws** (e.g., deadlocks, inconsistencies).
 - Enables **prediction of non-functional properties** (performance, reliability, security) before implementation
 - Supports automated checks and simulations

ADL

- **Task 4: Improve Documentation and Maintainability**
- Informal documentation quickly becomes outdated, leading to lost architectural knowledge and increased technical debt.
 - Provides a **formal, persistent record** of architectural decisions.
 - Ensures **accurate and up-to-date documentation** that evolves with the system.
 - Facilitates **knowledge transfer** to new team members.
 - Simplifies system evolution and maintenance over time.

ADL

- **Task 5: Support Automation and Model-Driven Development**
- Manual translation from design to code is error-prone and time-consuming.
 - Some ADLs enable **code generation** (e.g., skeleton code, configuration files).
 - Supports **automated deployment** and system configuration.
 - Promotes **consistency** between the architectural model and the implemented system.
 - Forms the basis for **Model-Driven Engineering (MDE)**.

ADL Assists in

- **Taming Complexity:** Manage large-scale system structures.
- **Clear Communication:** Bridge gaps between stakeholders.
- **Early Analysis:** Detect issues before coding.
- **Robust Documentation:** Ensure maintainability and knowledge retention.
- **Automation:** Facilitate efficient development and deployment.

ADLs transform architecture from art to **engineering discipline.**

Steps for drawing Software Architectural Diagram

- Software Architectural diagram Identify the three layers; Business layers, Application Layer and Technology/Physical Layers
 - In Business layers Identify the various ArchiMate Elements such as actors, business roles, collaborator, interfaces, processes, business functions, Interaction, event, services, object and products
 - For Application Layers identify components, collaborator, interfaces, processes, business functions, Interaction, event, services, object
 - For Technology Layer identify various elements which will be used for realizing
 - Draw appropriate connectors

Introduction

Structural (aka software architecture level of design)
issues include:

1. Organization of a system as a composition of components or Single unit

– **Monolithic** : implemented as a single unit

– **Advantages:**

- **Simplicity:** easy to develop, test, and debug.
- **Initial Development Speed:** A small team can build from scratch easily.
- **Easy Deployment:** self-contained, making deployment simple.
- **Cost Savings:** Can save money for small applications or organizations with limited resources.
- **Performance:** better performance due to no network latency and a single codebase
- **No external dependencies**

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

– Monolithic : implemented as a single unit

– **Limitations:**

- Slower development speed
- Inability to scale individual components
- Reliability issues due to tight coupling and dependencies
- Maintenance challenges as the codebase expands.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components or Single unit

- **Component based:** based on principle of modularity
- **Advantages:**
- **Reusability:** allows components to be incorporated into multiple projects eliminating the need to rewrite code for similar functionalities, saving development time and effort. Reusable components act as pre-built modules, readily available for integration into new applications.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components or Single units

- **Component based:** based on principle of modularity
- **Advantages:**
- **Extensibility:** designed to accommodate future growth and modifications which allows developers to add new functionalities to existing components without affecting their core functionality or impacting other components that rely on them.
- Ensures the system can evolve and adapt to changing requirements.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Advantages:**
- **Replaceability:** Designed for independent operation, enabling them to be easily replaced with alternative components that offer similar functionalities.
- Promotes flexibility and simplifies maintenance.
- If a component becomes outdated or requires an upgrade, it can be swapped out without disrupting the entire system as long as the replacement component adheres to the same interface.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Advantages:**
- **Independence:** Strive to operate independently, minimizing their reliance on external factors.
- Minimal dependencies on other components and can function with minimal external resources.
- Promotes maintainability and simplifies deployment, as components can be easily integrated into different environments with minimal configuration changes.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Advantages:**
- **Encapsulation:** internal logic and data, hiding implementation details from other components.
- Only expose their functionalities through well-defined interfaces promoting loose coupling, where changes within a component have minimal impact on other components that interact with it.
- Simplifies maintenance and testing by isolating functionalities within each component.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Benefits:**
 - Faster Development time
 - Reduced Cost
 - Improved Scalability
 - Better Quality

Introduction

Structural (aka software architecture level of design) issues include:

- 1. Organization of a system as a composition of components or single unit**
 - **Component based:** based on principle of modularity
 - **Challenges:**
 - **Decomposition Complexity:** Breaking down into well-defined components requires careful planning and upfront investment.
 - Involves identifying functional boundaries, ensuring components have clear responsibilities, and minimizing dependencies between them.
 - Extensive applications with tightly coupled functionalities can be particularly challenging to decompose effectively.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Challenges:**
- **Granularity of Components:**
- Finding the right balance between reusability and specificity is crucial.
- Overly generic components require extensive customization to fit specific use cases, while highly specialized components have limited applicability across different projects.
- Developers need to carefully consider the trade-offs between reusability and the effort required for adaptation.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Challenges:**
- **Component Discovery and Selection**
- Locating suitable pre-built components can be a time-consuming process.
- Developers need to evaluate available component libraries or repositories, considering factors like functionality, compatibility, documentation quality, and maintenance status.
- If suitable components do not exist, requiring developers to build them from scratch, negating some of the reusability benefits of CBA.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Challenges:**
- **Integration and Testing Challenges**
- Linking numerous components into a cohesive system requires careful consideration of interactions and dependencies.
- Interface compatibility and data exchange formats need to be carefully defined to ensure seamless communication.
- Testing becomes more complex as it involves not just individual components but also their interactions within the larger system.

Introduction

Structural (aka software architecture level of design) issues include:

1. Organization of a system as a composition of components

- **Component based:** based on principle of modularity
- **Challenges:**
- **Monitoring and Maintenance Overhead**
- Managing a system composed of multiple components introduces additional monitoring requirements.
- Developers need to track the performance and health of individual components, as well as their interactions with each other.
- Keeping component libraries up-to-date with security patches and bug fixes can add complexity, especially for large-scale systems with numerous components.

Introduction

Structural (aka software architecture level of design)
issues include:

2. Global control structures:

- Freely accessible data will allow many parts of the software system to access the data
- Hamper the security of the data as it could be easily accessed/ modified by any part of the system making the system vulnerable and difficult to test and maintain

e.g. global data in C program as compared to data protection in C++

Introduction

Structural (aka software architecture level of design)
issues include:

3. Protocols of communication:

- If different parts/ components are used to implement a system their compatibility with each other becomes a crucial point
- The interfaces between the components play vital role
- External dependency will increase affecting the latency period
- Some changes in structure of an external components may affect the system under consideration/ development

Introduction

Structural (**aka software architecture level of design**) issues include:

4. Synchronization and Data access

- Performance / speed of different components may affect the performance of system
 - e.g. the speed of CPU is always higher than memory/ peripheral devices affecting overall performance of the system
 - The internet speed always control the performance of distributed system.
- Data accessed by *OTHER external* components may compromise the security, correctness

Introduction

Structural (aka software architecture level of design)
issues include:

- 5. The assignment of functionality to design elements**
 - Decision about distribution of work across components will require experience
 - Choosing the correct components is very crucial
 - Availability of components at the time of designing and using

Introduction

Structural (aka software architecture level of design)
issues include:

6. The compositions of design elements

- How individual, self-contained building blocks of a software system are combined, arranged, and interconnected to form a cohesive and functional whole
- Distribution of components
 - Local
 - Developed by own
 - Ready to use (with modification) from existing
 - Off The Shelf
 - Software as Service

Introduction

Structural (**aka software architecture level of design**) issues include:

7. Physical distribution

- how the various components, services, and data stores of a software system are deployed and distributed across actual hardware infrastructure, network nodes, and geographical locations

- **Deployment Topologies:**

- Centralized:
- Distributed:
- Client-Server:
- Peer-to-Peer (P2P)
- N-Tier (Multi-Tier).
- Microservices
- Event-Driven Architectures
- Cloud Deployments: [Module 1 SADT \(2025-26\)](#)

Introduction

Structural (aka software architecture level of design) issues include:

7. Physical distribution

- **Deployment Topologies:**

- **Centralized:** All components run on a single server or a cluster of closely co-located servers. (e.g., a monolithic application deployed on one machine).
- **Distributed:** Components are spread across multiple machines, potentially in different data centers or cloud regions. This is the more common scenario for modern, scalable applications.
- **Client-Server:** A clear separation between a client (user interface) and a server (business logic, data).

Introduction

Structural (aka software architecture level of design) issues include:

7. Physical distribution

- **Deployment Topologies:**

- **Peer-to-Peer (P2P):** Each node can act as both a client and a server.
- **N-Tier (Multi-Tier):** Logical layers (presentation, business, data) are deployed on separate physical machines or clusters.
- **Microservices:** Individual services are deployed independently, often in containers (e.g., Docker) and managed by orchestration platforms (e.g., Kubernetes).

Introduction

Structural (aka software architecture level of design) issues include:

7. Physical distribution

- **Deployment Topologies:**

- **Event-Driven Architectures:** Components communicate primarily through events, often using message brokers, allowing for highly decoupled and scalable deployments.
- **Cloud Deployments:** Leveraging cloud providers (AWS, Azure, GCP) to deploy components as virtual machines, containers, serverless functions, or managed services.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

- Monolithic Architecture (Lack of Modularity/Loose Coupling)
- Shared State and Global Locks
- Database Bottlenecks (Single Relational Database Instance)
- Synchronous Communication and Chained Dependencies
- Tight Coupling to Specific Technologies/Vendors
- Inefficient Data Access Patterns / Lack of Caching Strategy
- No Clear Bounded Contexts / Domain Separation (Microservices Specific)

Introduction

Structural (aka software architecture level of design) issues include:

8. Scaling

Monolithic Architecture (Lack of Modularity/Loose Coupling)

- **Vertical Scaling Only (often):**
- It's difficult to scale individual components independently. If one small feature (e.g., user authentication) becomes a bottleneck, you have to scale the entire application, which is inefficient and costly.
- **Resource Inefficiency:** Different parts of the monolith might have vastly different resource requirements. Scaling the whole application means over-provisioning resources for parts that don't need them.

Introduction

Structural (aka software architecture level of design) issues include:

8. Scaling

- **Deployment Bottlenecks:** A small change requires redeploying the entire application, leading to downtime or complex blue-green deployments for large systems.
- **Development Bottlenecks:** Large teams working on a single codebase can lead to merge conflicts and slower development cycles

Introduction

Structural (aka software architecture level of design) issues include:

8. Scaling

Shared State and Global Locks

- **Concurrency Bottlenecks:** Global locks become contention points, serialized access, and severely limit parallel processing, even if you add more compute resources.
- **Cache Invalidation Complexity:** If shared caches are not distributed or managed properly, consistency issues arise, or invalidation becomes a complex scaling challenge.
- **Increased Latency:** Waiting for locks or distributed consensus can add significant latency.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Database Bottlenecks (Single Relational Database Instance)

Relying on a single, centralized relational database instance for all data storage and retrieval, especially with complex joins or heavy write loads

- **Vertical Scaling Limits:.**
- **Write Contention: Read Replicas Limitations:**
- **Schema Rigidity:**
- **Lack of Sharding/Partitioning**

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Database Bottlenecks (Single Relational Database Instance)

- **Vertical Scaling Limits:**
 - Relational databases can only scale vertically (more CPU, RAM, faster disk) to a certain extent before hitting physical limits.
- **Write Contention:**
 - Heavy write loads can saturate a single database instance.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Database Bottlenecks (Single Relational Database Instance)

- **Read Replicas Limitations:**
 - While read replicas help with read scaling, writes remain a bottleneck on the primary.
- **Schema Rigidity:**
 - Changing schemas on a large, single database can be complex and risky, hindering agile development needed for evolving systems.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Database Bottlenecks (Single Relational Database Instance)

- **Lack of Sharding/Partitioning:**
 - Sharding: horizontal partitioning, is a method of distributing a large database across multiple servers to improve performance and scalability
 - If the architecture doesn't support data partitioning or sharding from the outset, retrofitting it can be extremely difficult and costly.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Synchronous Communication and Chained Dependencies

Components strictly relying on synchronous request-response interactions, especially in long chains

- **Cascading Failures**
- **Latency Accumulation**
- **Limited Throughput**
- **Difficulty with Load Spikes**

Introduction

Structural (aka software architecture level of design) issues include:

8. Scaling

Synchronous Communication and Chained Dependencies

Components strictly relying on synchronous request-response interactions, especially in long chains

- **Cascading Failures:**

- A failure in one component can block or cause failures in upstream components, propagating through the system and potentially bringing down the entire service.

- **Latency Accumulation:**

- The total response time is the sum of all individual service latencies in the chain, limiting overall throughput.

Introduction

Structural (aka software architecture level of design) issues include:

8. Scaling

Synchronous Communication and Chained Dependencies

Components strictly relying on synchronous request-response interactions, especially in long chains

- **Limited Throughput:**

- Each request ties up resources until the entire chain completes, limiting the number of concurrent requests the system can handle.

- **Difficulty with Load Spikes:**

- A sudden spike in requests can quickly overwhelm downstream services if they don't scale instantly or have built-in backpressure mechanisms.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Tight Coupling to Specific Technologies/Vendors

Tight Coupling to Specific Technologies/Vendors:

- Limited Options**
- Vendor Lock-in**
- Migration Difficulty**

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Tight Coupling to Specific Technologies/Vendors

– Limited Options:

- Restricts the ability to leverage alternative technologies that might offer better scaling characteristics for specific workloads.

– Vendor Lock-in:

- Makes it difficult to move to a more cost-effective or performant platform if scaling needs dictate.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

Tight Coupling to Specific Technologies/Vendors

– Migration Difficulty:

- Retrofitting new technologies for scaling (e.g., switching from a relational DB to NoSQL for certain workloads) becomes a massive refactoring effort.

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

- **Inefficient Data Access Patterns / Lack of Caching Strategy:**
- Frequent, direct database calls for commonly accessed data, or lack of proper caching layers at various levels (application, distributed cache, CDN (Content Delivery Network) to the practice of dividing a large dataset into smaller, more manageable pieces, called shards, and distributing them across multiple CDN servers or locations)
- **Database Overload**
- **Increased Latency.**
- **Redundant Computations**

Introduction

Structural (aka software architecture level of design)
issues include:

8. Scaling

- **Inefficient Data Access Patterns / Lack of Caching Strategy:**
- **Database Overload:**
 - Every read hits the database, leading to bottlenecks even with read replicas.
- **Increased Latency:**
 - Retrieving data from the primary data store is inherently slower than from a cache.
- **Redundant Computations:**
 - Recalculating or re-fetching data that could be cached.

Introduction

Structural (aka software architecture level of design)
issues include:

9. Performance

- when the fundamental design and organization of the system inherently limit its ability to achieve desired performance levels, regardless of code optimization or hardware upgrades
- **Single Points of Contention:**
 - If the architecture funnels all critical operations through a single component (e.g., a single database instance for all writes, a single message queue, or a global lock), this component will inevitably become a bottleneck as load increases, even if its code is perfectly optimized. This is a design choice that limits concurrency and throughput.

Introduction

Structural (aka software architecture level of design) issues include:

9. Performance

- **Synchronous Chains:**

- An architecture that relies heavily on long, synchronous call chains means that the performance of the entire operation is dictated by the slowest link in the chain. Adding more resources might not help if one service in the sequence is inherently slow or prone to latency.

- **Excessive Network Hops:**

- If architectural elements are physically distributed (e.g., microservices), but the communication patterns require an excessive number of network calls between them for a single operation, network latency can become a dominant factor.

Introduction

Structural (aka software architecture level of design) issues include:

9. Performance

- **Serialization/Deserialization Overheads:**
 - Frequent data transfer between services often involves serialization and deserialization (e.g., JSON, XML). If the data payloads are large or the process is inefficient, this architectural choice can introduce significant overhead.
- **Centralized Data Storage for Distributed Systems:**
 - If a highly distributed system still relies on a single, centralized database that cannot scale horizontally (e.g., a traditional RDBMS pushed beyond its vertical scaling limits), the database becomes the performance bottleneck. This is a structural choice about data management.

Introduction

Structural (aka software architecture level of design) issues include:

9. Performance

- **Lack of Caching Strategy:**
 - An architecture that doesn't account for caching at various layers (client-side, application-level, distributed caches, CDNs(Content Delivery Networks)) will force every request to hit the primary data store, leading to high latency and database load. This is a missing structural component.
- **Inefficient Data Partitioning/Sharding:**
 - If data is not partitioned or sharded effectively across multiple database instances, certain hot spots or queries might still hit a single server, leading to performance degradation. The sharding strategy is an architectural decision.

Introduction

Structural (aka software architecture level of design) issues include:

10. Dimensions of evolution

– the various aspects or axes along which a software system can and often must change over its lifetime

- **Changing Requirements:**

– User needs, business goals, and regulations shift.

- **Technological Advancements:**

– New frameworks, languages, and infrastructure emerge.

- **Performance and Scalability Demands:**

– Growth in users or data requires more robust solutions.

- **Security Threats:**

– New vulnerabilities necessitate architectural adjustments.

- **Maintenance and Bug Fixes:**

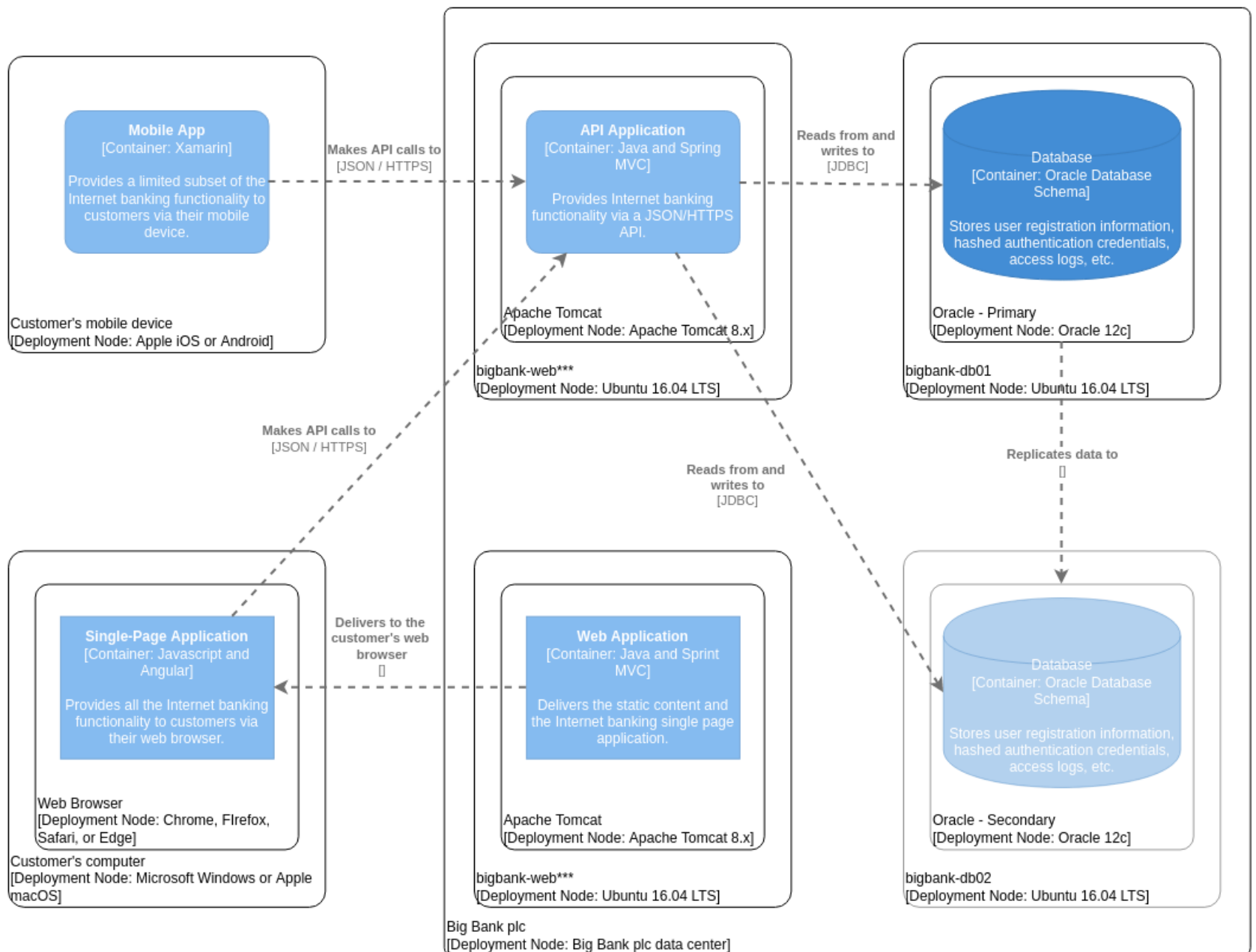
– Identifying and resolving issues can lead to design improvements.

Introduction

Structural (**aka software architecture level of design**) issues include:

11. Selection among design alternatives

- It's rarely about finding a single "perfect" solution, but rather about making informed trade-offs to achieve the optimal balance of competing concerns for a given context
- **Styles:** Monolithic, Microservices, Event-Driven, Layered, Client-Server, Serverless, Space-Based, etc.
- **Technology Stacks:** Different programming languages, frameworks, databases, cloud platforms.
- **Deployment Models:** On-premise, public cloud, private cloud, hybrid cloud.
- **Integration Patterns:** Message queues, REST APIs, GraphQL, gRPC.
- **Data Storage Solutions:** Relational databases, NoSQL databases, data lakes.



Introduction

Software Architecture involves:

- the description of elements from which system are built
- Interactions among those elements
- Patterns that guide their composition
- Constraints on those patterns
- The relative informality and high level of abstraction in describing architecture helps software engineers to understand the system implementation
- Architecture provides abstract relation detail of the actual computations of the elements providing natural framework of understanding broader , system-level concerns
- Provides the **skeleton** around which system can be built

Software Architecture in context of Software Engineering

- Typical Software Engineering phases and activities:
 - **Requirements:** the statement of **what** the application is supposed to do. Typically requirement analysis and specification is that activity resulting requirements document which becomes basis for design which might satisfy the identified requirements.

Design Decisions

The Design Decision / Recommendation:

- Providing an overview of decision highlighting the key expected or known implications for the operating model, data, technology, process & people, and how it supports the approved business outcomes and design principles.
- Key guidelines to achieve non-functional requirements of software systems in the early stages of software development.

Design Decisions

The Design Decision / Recommendation:

- Which *components* are to be used?
- How the various *components* are going to be working *together*?
- Can existing *components* be used as it is, or is modification needed?
- Are the components to be developed (built) or can ready to use (of-the-shelf) components be used?

Design Decisions

Encompass every aspect of system under development related to:

- **System structure:** Organization of the elements
- **Functional behavior:** Sequencing of data processing, storage and visualization
- **Interaction:** communication between all elements
- **Non-functional properties:** system dependability, reliability, scalability etc. will be ensured
- **Implementation:** how the elements will be implemented

Basic Concepts

SA involves:

- the description of elements from which system are built
- interactions among those elements
- patterns that guide their compositions
- constraints on these patterns

A system will be defined in terms of collection of components and interactions among them, which may become part of another bigger system

Basic Concepts

SA will be :

- Abstractly represented as box- and –line diagrams accompanying details explaining the meaning of each symbol specifying choice of each component

Basic Concepts

SA involves:

- the description of elements from which system are built
- interactions among those elements
- patterns that guide their compositions
- constraints on these patterns

A system will be defined in terms of collection of components and interactions among them, which may become part of another bigger system

Temporal Aspect

- Design decisions are and made over a system's lifetime
- The "temporal aspect" of software architecture refers to how a system's design considers and manages *time* and *change* throughout its lifecycle.
- It's not just about the current state of the system, but how it behaves, evolves, and interacts with events and data *over time*

Temporal Aspect

- Design decisions are and made over a system's lifetime
- The "temporal aspect" of software architecture refers to how a system's design considers and manages *time* and *change* throughout its lifecycle.
- It's not just about the current state of the system, but how it behaves, evolves, and interacts with events and data *over time*
→ Architecture has a temporal aspect

Importance of Temporal Aspect

- **Correctness:**
 - For real-time systems, temporal correctness is as important as logical correctness. For historical data, it ensures accurate auditing and reporting.
- **Robustness and Reliability:**
 - Designing for time-related failures (timeouts, network delays, component crashes) makes systems more resilient.
- **Business Agility:**
 - An architecture that embraces temporal considerations and evolution allows the business to adapt faster to changing market conditions and regulatory demands.

Importance of Temporal Aspect

- **Maintainability and Longevity:**
 - Systems designed to manage their own temporal changes are easier and less costly to maintain over their lifespan.
- **Risk Mitigation:**
 - Addressing temporal concerns upfront reduces the risk of major architectural rework, data inconsistencies, or missed deadlines later on.

Basic Concepts

Perspective Architecture (design decision set P):

- Recommendation about how the system *should be* implemented
- As-intended or as- conceived architecture
- Need not exist in any tangible form
- The set of decisions P will change over time and *must* be recorded as revised and *realized* artifacts presented as set A
- Artifacts could be refinements of architectural design decisions in UML format or different architectural styles and patterns

Basic Concepts

Perspective architecture v/s Descriptive architecture:

- As intended – as conceived architecture
- Perspective architecture **need not** necessarily exist in any tangible form **P**
- Descriptive architecture is representing it in any form (UML etc.) **D**

As-Designed vs. As-Implemented Architecture

- Which architecture is “correct”?
- Are the two architectures consistent with one another?
- What criteria are used to establish the consistency between the two architectures?
- On what information is the answer to the preceding questions based?

Architectural Evolution

- When a system evolves, ideally its prescriptive architecture is modified first
- In practice, the system – and thus its descriptive architecture – is often directly modified
- This happens because of
 - Developer sloppiness
 - Perception of short deadlines which prevent thinking in detail and documenting
 - Lack of documented prescriptive architecture
 - Need or desire for code optimizations
 - Inadequate techniques or tool support

Basic Concepts

Architectural Degradation:

- **P** and **D** would be identical at the start but over the period there could be changes while realizing the architecture
- When a system is initially developed or when already implemented is evolved , **P** is modified

Architectural Degradation

- Two related concepts
 - Architectural drift
 - Architectural erosion
- *Architectural drift* is introduction of principal design decisions into a system's descriptive architecture that
 - are not included in, encompassed by, or implied by the prescriptive architecture
 - but which do not violate any of the prescriptive architecture's design decisions
- *Architectural erosion* is the introduction of architectural design decisions into a system's descriptive architecture that violate its prescriptive architecture

Architectural Recovery

- If architectural degradation is allowed to occur, one will be forced to *recover* the system's architecture sooner or later
- *Architectural recovery* is the process of determining a software system's architecture from its implementation-level artifacts
- Implementation-level artifacts can be
 - Source code
 - Executable files
 - Java .class files

Deployment

- A software system cannot fulfill its purpose until it is *deployed*
 - Executable modules are physically placed on the hardware devices on which they are supposed to run
- The deployment view of an architecture can be critical in assessing whether the system will be able to satisfy its requirements
- Possible assessment dimensions
 - Available memory
 - Power consumption
 - Required network bandwidth

Software Architecture's Elements

- A software system's architecture typically is not (and should not be) a uniform monolith
- A software system's architecture should be a composition and interplay of different elements
 - Processing
 - Data, also referred as information or state
 - Interaction

Components

- Elements that encapsulate processing and data in a system's architecture are referred to as *software components*
- Address key system concerns such as:
 - Processing / functionality/ behavior
 - State: information or data
 - Interaction: interconnection, communication, coordination
- Components typically provide application-specific services

Components

Definition

- A *software component* is an architectural entity that
 - encapsulates a subset of the system’s functionality and/or data
 - restricts access to that subset via an explicitly defined interface
 - has explicitly defined dependencies on its required execution context
- Component is a locus computation and state in a system
- Can be as simple as an operation or as complex as an entire system depending on perspective
- Can be “seen” by the users (software or **human**) from outside through interface or its like a “black box”
- Components implements software engineering principles such as encapsulation, abstraction and modularity

Components

- Usable and reusable across applications
- Extent of the context by components include:
 - Component's required interface to services provided by other components in a system on which component depends for its ability to perform its operation
 - Availability of resources, such as data files or directories on which the components relies
 - Required system software , such as programming language run time environments, middleware platforms, operating systems, network protocols and device drivers
 - Hardware configuration needed o execute the components

Connectors

- In complex systems *interaction* may become more important and challenging than the functionality of the individual components
- **Definition**
 - A *software connector* is an architectural building block tasked with effecting and regulating interactions among components
- In many software systems connectors are usually simple procedure calls or shared data accesses
 - Much more sophisticated and complex connectors are possible!
- **Connectors typically provide application-independent interaction facilities**

Examples of Connectors

- Shared data access:
 - nonlocal variables or shared memory
 - Allow multiple software components to interact by reading from and writing to the shared facilities
 - Interaction is distributed in time
- Distribution connectors:
 - Encapsulate network library APIs
 - Usually coupled with more basic connectors
 - E.g. remote procedure call (RPC)
- Adaptor connectors –stdio.h

Examples of Connectors

- Procedure call connectors - `mod(a,b)`
- Message passing connectors
- Streaming connectors – `printf()`
- Distribution connectors
- Wrapper/adaptor connectors – `stdio.h`

Configurations

- Components and connectors are composed in a specific way in a given system's architecture to accomplish that system's objective
- **Definition**
 - An *architectural configuration*, or topology, is a set of specific associations between the components and connectors of a software system's architecture

Architectural Models, Views, and Visualizations

- Architecture Model
 - An artifact documenting some or all of the architectural design decisions about a system
- Architecture Visualization
 - A way of depicting some or all of the architectural design decisions about a system to a stakeholder
- Architecture View
 - A subset of related architectural design decisions

Architectural Processes

- Architectural design
- Architecture modeling and visualization
- Architecture-driven system analysis
- Architecture-driven system implementation
- Architecture-driven system deployment, runtime redeployment, and mobility
- Architecture-based design for non-functional properties, including security and trust
- architectural adaptation

Stakeholders in a System's Architecture

- Stakeholders are individuals, groups, or organizations that are actively involved in a software project, can influence it due to their position, and whose interests may be affected by the success or failure of the project.
 - Architects
 - Developers
 - Testers
 - Managers
 - Customers
 - Users
 - Vendors

1.2 Styles and Architectural Patterns

- Pipes and Filters
- Event Based

- 1.3

- Implicit Invocation
- Layered Systems
- Repositories
- Interpreters

Architectural Styles

- Certain design choices regularly result in solutions with superior properties
 - Compared to other possible alternatives, solutions such as this are more elegant, effective, efficient, dependable, evolvable, scalable, and so on
- **Definition**
 - An *architectural style* is a named collection of architectural design decisions that
 - are applicable in a given development context
 - constrain architectural design decisions that are specific to a particular system within that context
 - elicit beneficial qualities in each resulting system

Architectural Patterns

- **Definition**

- An *architectural pattern* is a set of architectural design decisions that are applicable to a recurring design problem, and parameterized to account for different software development contexts in which that problem appears

- A widely used pattern in modern distributed systems is the *three-tiered system* pattern

- Science
 - Banking
 - E-commerce
 - Reservation systems

Three-Tiered Pattern

- Front Tier
 - Contains the user interface functionality to access the system's services
- Middle Tier
 - Contains the application's major functionality
- Back Tier
 - Contains the application's data access and storage functionality

Architectural Styles

- An established, shared understanding of common forms of design is one of the features of mature engineering field.
- Architectural levels could be client-server systems, a pipe-filter design or a layered architecture.
- Typically used design methods and notations are:
 - Object –Oriented
 - Dataflow organizations
- At a more detailed level, different styles will be used for a common framework, which will adopt architectural specific system as a **collection of components** (clients, servers, filters, layers, databases) with a description of **interactions using connectors** (procedure call, event broadcast, database protocols, pipes)

Architectural Styles

Common architectural styles:

- **Dataflow systems:**
 - Batch sequential
 - Pipes and filters
- **Call- and – return systems:**
 - Main program and subroutine
 - OO systems
 - Hierarchical layers
- **Independent Components:**
 - Communicating processes
 - Event systems
- **Virtual machines:**
 - Interpreters
 - Rule-based systems
- **Data-centered systems (repositories):**
 - Databases
 - Hypertext systems
 - Blackboards

Architectural Styles

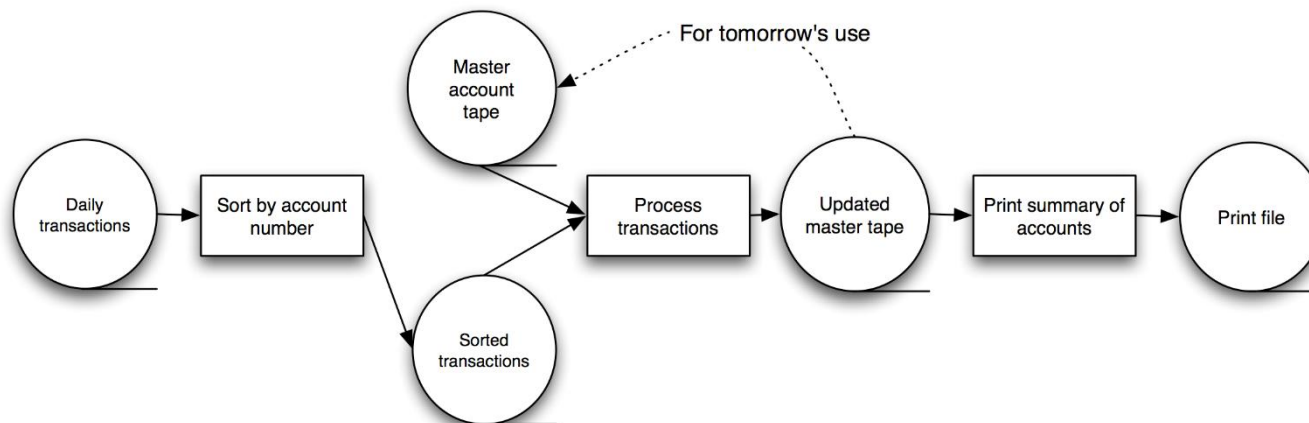
Common architectural styles:

- **Dataflow systems:**
 - **Batch sequential**
 - The age old styles
 - Separate programs are executed in order; data is passed as an aggregate from one program to the next.
 - Connectors: “The human hand” carrying tapes between the programs, a.k.a. “sneaker-net ”
 - Data Elements: Explicit, aggregate elements passed from one component to the next upon completion of the producing program’s execution.
 - Typical uses: Transaction processing in financial systems.

Architectural Styles

Common architectural styles:

- **Dataflow systems:**
 - **Batch sequential**



Architectural Styles

Dataflow systems:

Pipes and Filters

Components are filters

Transform input data streams into output data streams

Possibly incremental production of output

Connectors are pipes

Conduits for data streams

Style invariants

Filters are independent (no shared state)

Filter has no knowledge of up- or down-stream filters

Examples

UNIX shell

signal processing

Distributed systems

parallel programming

Example: `ls invoices | grep -e August | sort`

Architectural Styles

Dataflow systems:

Pipes and Filters

Variations

Pipelines — linear sequences of filters

Bounded pipes — limited amount of data on a pipe

Typed pipes — data strongly typed

Architectural Styles

Dataflow systems:

Pipes and Filters

Advantages

- System behavior is a succession of component behaviors
- Filter addition, replacement, and reuse
 - Possible to hook any two filters together
- Certain analyses
 - Throughput, latency, deadlock
- Concurrent execution

Disadvantages

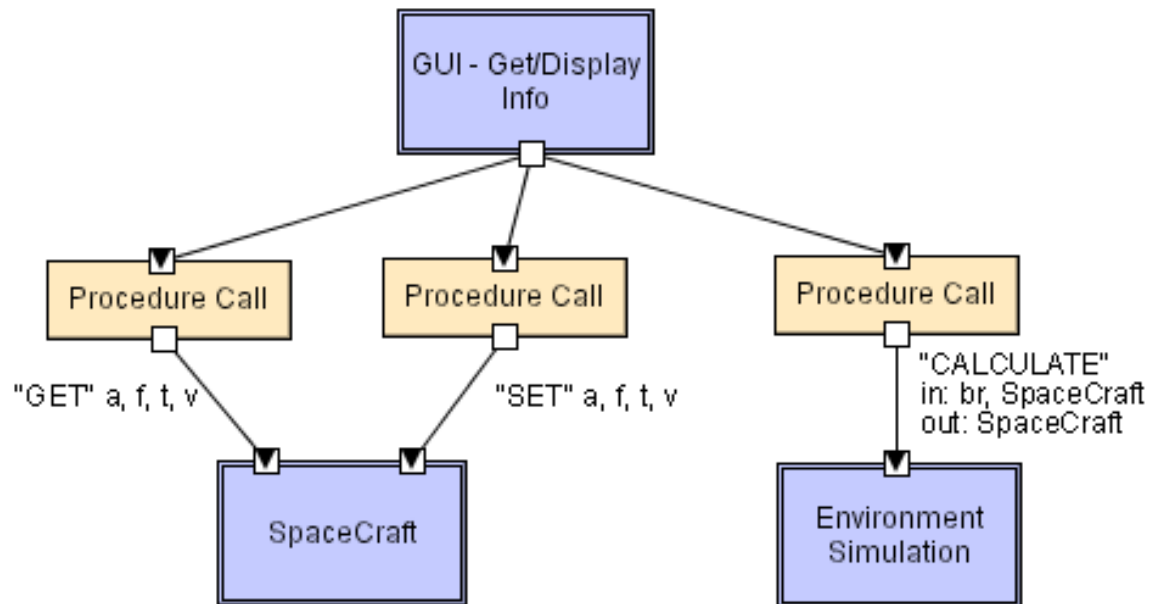
- Batch organization of processing
- Interactive applications
- Lowest common denominator on data transmission

Architectural Styles

- **Call- and – return systems:**
 - **OO Style**
- Components are objects
 - Data and associated operations
- Connectors are messages and method invocations
- Style invariants
 - Objects are responsible for their internal representation integrity
 - Internal representation is hidden from other objects
- Advantages
 - “Infinite malleability” of object internals
 - System decomposition into sets of interacting agents
- Disadvantages
 - Objects must know identities of servers
 - Side effects in object method invocations

Architectural Styles

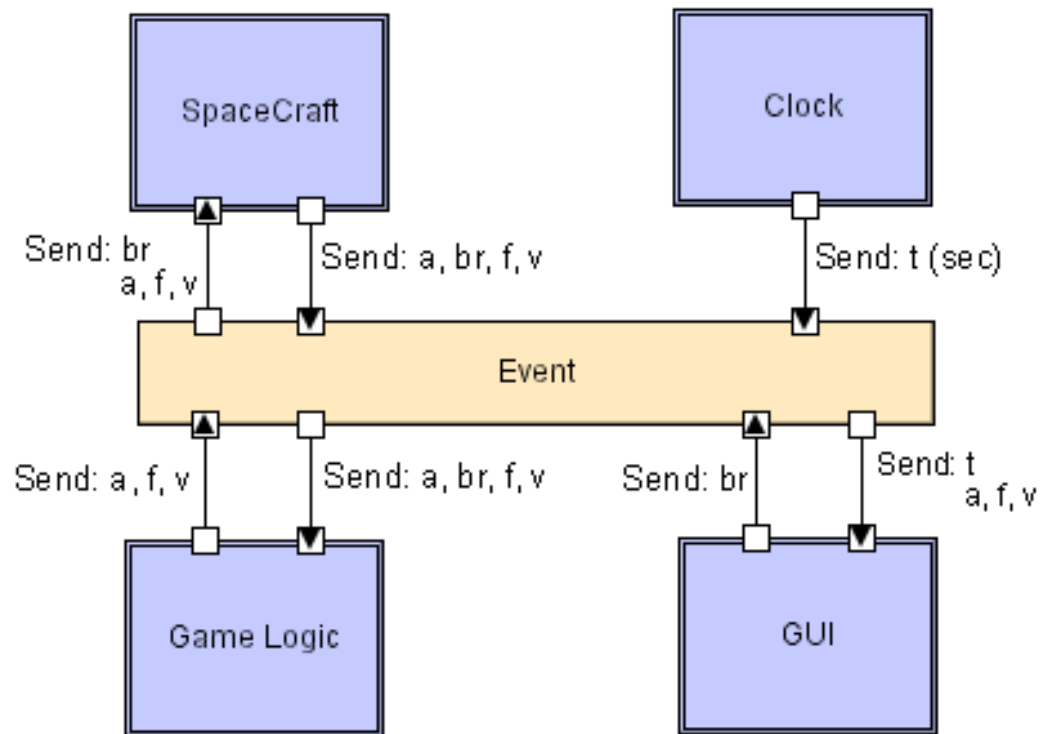
- Call- and – return systems:
 - OO Style



Event based

- Independent components asynchronously emit and receive events communicated over event buses
- Components: Independent, concurrent event generators and/or consumers
- Connectors: Event buses (at least one)
- Data Elements: Events – data sent as a first-class entity over the event bus
- Topology: Components communicate with the event buses, not directly to each other.
- Variants: Component communication with the event bus may either be push or pull based.
- Highly scalable, easy to evolve, effective for highly distributed applications.

Event based



Architectural Styles

- **Hierarchical layers**
- Hierarchical system organization
 - “Multi-level client-server”
 - Each layer exposes an interface (API) to be used by above layers
- Each layer acts as a
 - *Server*: service provider to layers “above”
 - *Client*: service consumer of layer(s) “below”
- Connectors are protocols of layer interaction
- Example: operating systems
- *Virtual machine* style results from fully opaque layers

Architectural Styles

- **Hierarchical layers**

Advantages

Increasing abstraction levels

Evolvability

Changes in a layer affect at most the adjacent two layers

Reuse

Different implementations of layer are allowed as long as interface is preserved

Standardized layer interfaces for libraries and frameworks

Disadvantages

Not universally applicable

Performance

Layers may have to be skipped

Determining the correct abstraction level

Implicit invocation

Event announcement instead of method invocation

“Listeners” register interest in and associate methods with events

System invokes all registered methods implicitly

Component interfaces are methods and events

Two types of connectors

Invocation is either explicit or implicit in response to events

Style invariants

“Announcers” are unaware of their events’ effects

No assumption about processing in response to events

Implicit invocation

Advantages

- Component reuse

- System evolution

- Both at system construction-time & run-time

Disadvantages

- Counter-intuitive system structure

- Components relinquish computation control to the system

- No knowledge of what components will respond to event

Repositories

Blackboard:

Two kinds of components

- Central data structure — blackboard

- Components operating on the blackboard

System control is entirely driven by the blackboard state

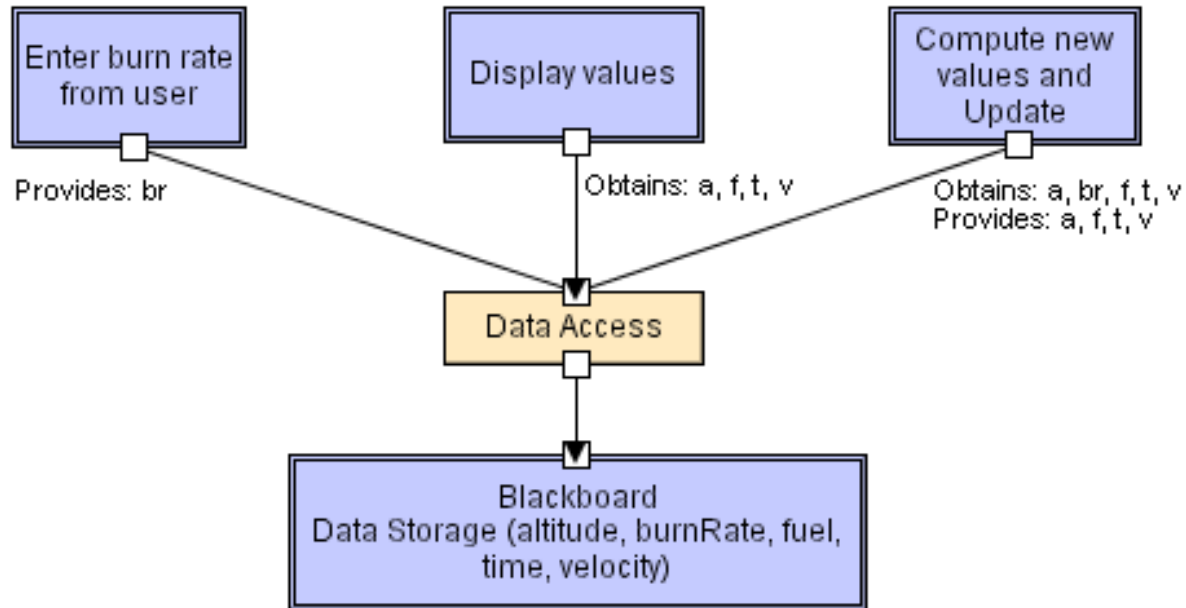
Examples

- Typically used for AI systems

- Integrated software environments

- Compiler architecture

Repositories



Interpreters

Interpreter parses and executes input commands, updating the state maintained by the interpreter

Components: Command interpreter, program/interpreter state, user interface.

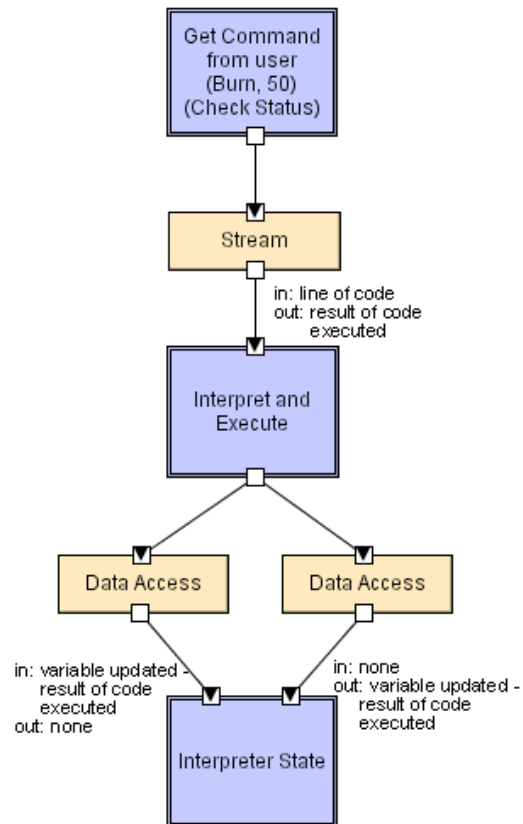
Connectors: Typically very closely bound with direct procedure calls and shared state.

Highly dynamic behavior possible, where the set of commands is dynamically modified. System architecture may remain constant while new capabilities are created based upon existing primitives.

Superb for end-user programmability; supports dynamically changing set of capabilities

Lisp and Scheme

Interpreters



References

- <https://www.geeksforgeeks.org/component-based-diagram/#steps-to-create-a-componentbased-diagram> accessed on July 4, 2024
- <https://datamyte.com/blog/component-based-architecture/> accessed on July 4, 2024
- <https://www.lucidchart.com/blog/how-to-design-software-architecture>
Accessed on July 5, 2024

1.4 Architectural Design Process

Software Architecture:

- The fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution. (IEEE 1471)
- The "big picture" view of the system under consideration.

1.4 Architectural Design Process

Software Architecture:

- Defines the high-level structure, behavior, and properties.
- Not just code; it's about concepts, decisions, and rationale , the underlying concepts, the choices made, and the reasoning behind those choices.
- It's about making crucial decisions early on that impact the entire lifecycle of the software.

1.4 Software Architectural Design Process

Purpose of Architectural Design

- Ensures structure
- Enhances maintainability
- Facilitates stakeholder communication
- Guides implementation

Characteristics of a Good Architecture

- Scalability
- Maintainability
- Performance
- Security
- Modularity
- Separation of concerns

Software Architecture Design Process

- Identify Architecture Styles
- Analyse Business Requirements
- Analyse Technical Requirements
- Design the Architecture
- Evaluate

Identify Architecture Styles

- Layered
- Client-server
- Micro-Services
- Event-driven
- Service-oriented

Analyze Business Requirements

- Stakeholder needs
- Functional goals
- Non-functional requirements

Analyze Technical Requirements

- Technology constraints
- Integration needs
- Deployment environment

Design the Architecture

- Select architecture style
- Define components
- Determine interactions
- Evaluate trade-offs

Evaluate the Architecture

- Scenario-based evaluation
- Performance modeling
- Risk analysis

Common Architectural Styles

- Monolithic
- Layered
- Client-server
- Event-driven
- Micro-Services

Layered Architecture

- Presentation Layer
- Business Logic Layer
- Data Access Layer
- Database

Client-Server Architecture

- Clients request services
- Server processes and responds

Microservices Architecture

- Decomposed into small services
- Independently deployable
- Communicate via APIs

Challenges in Architectural Design

- • Changing requirements
- • Technology evolution
- • Scalability needs
- • Resource constraints

Role of the Software Architect

- • Define vision
- • Lead technical design
- • Communicate with stakeholders
- • Ensure alignment with business goals

Best Practices for Architectural Design

- • Follow design principles
- • Document decisions
- • Involve stakeholders
- • Evaluate early

Tools and Techniques

- • UML diagrams
- • Architecture Decision Records (ADRs)
- • Design tools (e.g., Enterprise Architect)

Case Study Example

- • Online retail application
- • Apply layered + microservices
- • Discuss trade-offs

Summary

- • Architecture guides implementation
- • Involves requirement analysis, design, evaluation
- • Key for successful systems

References

- • Document sources
- • Books and research papers
- • Online resources

Questions?

- Open floor for audience questions and discussion.

Architectural Design Importance

- **Risk Mitigation:**
 - Addresses critical issues early.
 - Identifying and addressing potential show-stopping issues at the design stage is far less costly than fixing them during development or, worse, after deployment.
- **Stakeholder Communication:**
 - Provides a common understanding
 - A visual representation that everyone can understand, from developers to business owners

Architectural Design Importance

- **System Quality Attributes:**
 - Good architecture ensures quality attributes are built in as inherent properties and not added on separately
 - **Some Quality attributes are:**
 - Performance, security, scalability, reliability, usability, security, and especially performance and scalability.

Architectural Design Importance

- **Reusability & Maintainability:**
 - Promotes modularity and clarity
 - by clear separation of concerns and modularity
- **Development Efficiency:**
 - Guides implementation and reduces rework
 - A clear roadmap for implementation, reducing ambiguity
- **Long-term Vision:**
 - Supports future evolution
 - Future requirements

Architectural Design Process

- **An iterative and incremental process.**
- **Not a linear or one-time activity.**
- **Driven by requirements and constraints**
 - both functional and non-functional – and various constraints, such as budget, technology stack, and time
- **Involves trade-offs and decision-making**
 - no one-size-fits-all solution, and it's about choosing the best path given the specific context
 - Build , Off The Shelf , Out source
- **Continuously refined and validated** throughout the project life cycle

Architectural Design Process

- Phase 1: Requirements Gathering & Analysis (Architectural View)
- Phase 2: Architectural Synthesis / High-Level Design
- Phase 3: Architectural Analysis & Evaluation
- Phase 4: Architectural Refinement & Documentation
- Phase 5: Architecture Communication & Governance

Architectural Design Process

Phase 1: Requirements Gathering & Analysis (Architectural View)

- a deep dive into requirements
- **Focus on**
 - Extracting architecturally significant requirements

Architectural Design Process

Phase 1: Requirements Gathering & Analysis (Architectural View)

- **Functional Requirements:** What the system *does*.
- **Non-Functional Requirements (NFRs) / Quality Attributes:**
 - Are often the true drivers of architectural decisions. These are the 'how well' aspects – how fast, how secure, how scalable the system needs to be.
 - Performance, Scalability, Security, Reliability, Usability, Maintainability, Portability, Testability, etc.

Architectural Design Process

Phase 1: Requirements Gathering & Analysis (Architectural View)

- **Constraints:**
 - Clearly identify any **constraints** that will limit our design choices
 - Computation at local end v/s remote computing
 - Storage at local drives or cloud
 - Budget, timeline, technology, regulatory
- **Key Deliverable of Phase 1:** Documented NFRs and Constraints.

Architectural Design Process

- Phase 2: Architectural Synthesis / High-Level Design
- **Objective of Phase 2:**
- Translate requirements into a conceptual structure using:
 - **Identify Major Components**
 - **Define Interfaces**
 - **Determine Communication Mechanisms**
 - **Select Architectural Styles/Patterns:**
 - **Preliminary Technology Stack Consideration:**
- **Key Deliverable:** Conceptual Architecture Diagram, Architecture Description.

Architectural Design Process

- **Phase 2: Architectural Synthesis / High-Level Design**
 - This is where the actual design begins.
 - In **architectural synthesis**, transform those abstract requirements into a concrete, high-level structure.
- **Identify Major Components:**
 - Logical separation of concerns
 - not classes or functions yet, but large, logical blocks that represent distinct responsibilities.
- **Define Interfaces:**
 - How components interact.
 - how they will communicate with each other, and the **communication mechanisms** like REST APIs, message queues, or direct calls

Architectural Design Process

- **Phase 2: Architectural Synthesis / High-Level Design**
- **REST API**
- Representational State Transfer Application Programming Interface
- It is a type of API that adheres to the architectural style used to design networked applications, especially web services.
- Utilize standard HTTP methods to interact with resources, typically exchanging data in formats like JSON or XML

Architectural Design Process

- **Phase 2: Architectural Synthesis / High-Level Design**
- **Determine Communication Mechanisms:**
 - Protocols, data formats
- **Select Architectural Styles/Patterns:**
 - Based on NFRs.
 - examples include client-server, microservices, or layered architecture
- **Preliminary Technology Stack Consideration:**
 - High-level choices
 - Based on the applications and requirements the technology stack will be selected

Architectural Design Process

- **Phase 2: Architectural Synthesis / High-Level Design**
- **Objective of Phase 2:**
- Translate requirements into a conceptual structure
- **Key Deliverable:** Conceptual Architecture Diagram, Architecture Description.

Architectural Design Process

- **Phase 3: Architectural Analysis & Evaluation**
- **Objective:** Assess the architecture against requirements.
- **Techniques:**
 - Architecture Reviews
 - Scenario-Based Evaluation (e.g., ATAM)
 - Prototyping/Proof of Concept
 - Simulation/Modeling
 - Identify Risks and Trade-off
- **Key Deliverable:** Feedback, identified risks, refined architecture.

Architectural Design Process

- **Phase 3: Architectural Analysis & Evaluation**
- **Architecture Reviews:**
 - Peer feedback, expert opinions
 - Compare various options and discuss with experts about merits and limitations of each
- **Scenario-Based Evaluation (e.g., ATAM)**
 - ATAM (Architecture Trade-off Analysis Method)
 - walking through specific use cases and challenging the architecture against key quality attributes like performance or security

Architectural Design Process

- **Phase 3: Architectural Analysis & Evaluation**
- **Prototyping/Proof of Concept:**
 - Rough idea about the product *before* it is actually implemented to **validate a design** choice
 - Saves resources which might have been wasted in developing unwanted/ wrong system
- **Simulation/Modeling**
 - help predict system behavior under stress
- **Identify Risks and Trade-off**
 - feedback loop is essential for refining the architecture

Architectural Design Process

- **Phase 3: Architectural Analysis & Evaluation**

Objective: Assess the architecture against requirements.

Key Deliverable: Feedback, identified risks, refined architecture.

Architectural Design Process

- **Phase 4: Architectural Refinement & Documentation**
- **Objective:** Refine and Document design
 - **Incorporate Feedback**
 - **Detailed Design of Key Components**
 - **Technology Stack Finalization**
 - **Deployment Architecture**
 - **Architectural Documentation**
- **Key Deliverable:** Comprehensive Architecture Document, Refined Diagrams.

Architectural Design Process

- **Phase 4: Architectural Refinement & Documentation**
- Based on the evaluation, move into **refinement** by iterating the process of design and solidifying the design
- **Incorporate Feedback:**
 - On received feedbacks, addressing identified risks and adjusting trade-offs redesign / refine the design
- **Detailed Design of Key Components:**
 - breaking down the components which will be leading to finalise the implementation

Architectural Design Process

- **Phase 4: Architectural Refinement & Documentation**
- **Technology Stack Finalization:**
 - choosing specific databases, message brokers, or frameworks
- **Deployment Architecture:**
 - Identify and document how the system will be deployed, scaled, and managed in its operational environment
- **Architectural Documentation:**
 - Prepare well-documented architecture using different viewpoints (like logical, process, physical views) for communication, future maintenance, and onboarding new team members.
 - Capture design decisions and their rationale.

Architectural Design Process

- **Phase 4: Architectural Refinement & Documentation**
- **Objective:** Refine and Document design
- **Key Deliverable:** Comprehensive Architecture Document, Refined Diagrams.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Aspects of Communication:**
- **Audience Identification:** Different stakeholders require different levels of detail and types of information.
 - **Development Teams:** Need technical specifications, design patterns, component interactions, API details, technology choices.
 - **Project Managers:** Need high-level overviews, resource implications, dependencies, risk assessments.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Aspects of Communication:**
 - **Business Stakeholders:** Need clarity on how the architecture supports business goals, scalability, performance, and future adaptability.
 - **Operations/DevOps:** Need deployment models, infrastructure requirements, monitoring strategies, resilience aspects.
 - **Security Teams:** Need security principles, vulnerability mitigation strategies, compliance considerations.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Message Tailoring:**
- **Level of Abstraction:**
 - Provide appropriate levels of detail for each audience.
Avoid overwhelming business users with technical jargon and avoid abstracting too much for developers.
- **Focus:**
 - Emphasize what's relevant to each group (e.g., performance for operations, user experience for product owners).

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Communication Channels & Artifacts:**
 - **Architectural Documentation:**
 - Not just monolithic documents, but targeted views (e.g., C4 model, ADRs, interface specifications, deployment diagrams, data models).
 - **Presentations & Walkthroughs:**
 - For initial architectural reviews, onboarding new team members, and presenting changes.
 - **Workshops & Whiteboarding Sessions:**
 - Collaborative design and problem-solving.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Message Tailoring:**
 - **Architectural Decision Records (ADRs):**
 - Documenting key architectural decisions, their rationale, alternatives considered, and consequences being crucial for historical context and future understanding.
 - **Regular Meetings:**
 - Architecture review meetings, technical sync-ups.
 - **Code Reviews:**
 - Ensuring adherence to architectural guidelines at the implementation level.
 - **Knowledge Repositories:** Wikis, Confluence, internal documentation sites.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
 - **Feedback Mechanisms:**
 - Provide channels for questions, concerns, and suggestions from development teams and other stakeholders.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Architecture Governance**
 - Establishing the processes, structures, and tools to ensure that the architecture remains sound, relevant, and aligned with organizational goals throughout its lifecycle.
 - It's about making sure the architecture is *actually used* and *evolves correctly*.

Architectural Design Process

- Phase 5: Architecture Communication & Governance
- **Architecture Governance**
- **Goals of Governance:**
 - Maintain architectural integrity and quality.
 - Ensure alignment with business strategy and enterprise architecture.
 - Manage technical debt and long-term maintainability.
 - Promote reusability and consistency across projects.
 - Mitigate risks related to system complexity, performance, scalability, and security.
 - Enable controlled evolution of the system over time.

Software Architecture

- The fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution.
- It's the "big picture" or blueprint of a software system.
- Deals with high-level design decisions.